Python Activity-Based Learning

Complete Collection with Flowcharts for Every Case Study

Combined 91 text/code files and 10 case-study flowcharts from the uploaded ZIP archive.

Python-Activity-Based-Learning/README.md

Python Activity-Based Learning (ABL)

Student name: Lingam Sasank Venkat

Roll number: 26000938

Branch: CSM

Section: 2

This repository contains 10 selected case studies from the faculty-provided Python ABL list. Each folder includes the problem statement, objectives, assumptions, algorithm, pseudocode, Python program, test case/output, and conclusion.

Selected case studies

1. Case Study 2: Progressive Electricity Billing
2. Case Study 11: Examination Result and Grace-Mark Validation
3. Case Study 13: ATM Transaction Sequence Simulation
4. Case Study 14: Warehouse Inventory Reconciliation
5. Case Study 16: Multi-Day Water Tank Simulation
6. Case Study 23: Student Timetable Conflict Detection
7. Case Study 27: Advanced Numeric Pattern Generator
8. Case Study 29: Password Policy Validation Engine
9. Case Study 42: Student Marks Matrix Analysis
10. Case Study 53: Food Waste Monitoring System

How to run

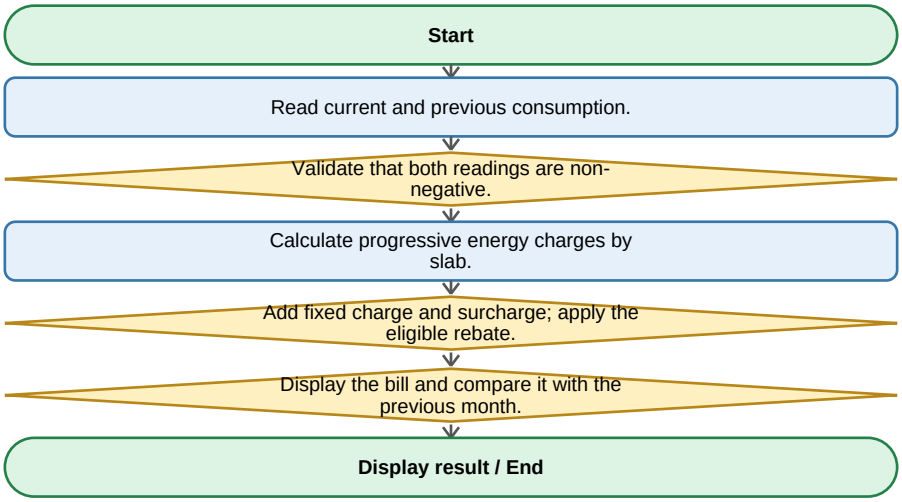
1. Install Python 3.
2. Open a terminal in a case-study folder.
3. Run `python program.py`.
4. Enter the sample inputs shown in `test_cases.md` where prompted.

Note: Tariff values, grace-mark rules, limits, and thresholds are sample assumptions for this academic exercise. Confirm any faculty-specific rules before final submission. Create flowchart images for each case study and add screenshots from your own execution before submitting.

Case Study 02: Progressive Electricity Billing

FLOWCHART

Progressive Electricity Billing



Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/problem_statement.md

Progressive Electricity Billing

Calculate a household's electricity bill using progressive consumption slabs, a fixed charge, surcharge, and rebate.

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/objectives.md

Objectives

- Calculate energy charges across multiple slabs.

- Apply fixed charges, surcharge, and rebate.

- Compare the current bill with the previous month's bill.

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/assumptions.md

Assumptions

- First 100 units cost ■2.00 per unit.
- Next 100 units cost ■3.00 per unit.
- Units above 200 cost ■5.00 per unit.
- Fixed charge is ■50; surcharge is 5% of energy charges; rebate is ■20 when consumption is at most 100 units.

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/algorithm.md

Algorithm

1. Read current and previous consumption.
2. Validate that both readings are non-negative.
3. Calculate progressive energy charges by slab.
4. Add fixed charge and surcharge; apply the eligible rebate.
5. Display the bill and compare it with the previous month.

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/pseudocode.txt

```
START
INPUT current_units, previous_units
IF either value is negative: DISPLAY error
ELSE
  CALCULATE energy charge by progressive slabs
  surcharge = energy charge * 5 / 100
  bill = energy charge + 50 + surcharge
  IF current_units &lt;= 100: bill = bill - 20
  DISPLAY energy charge, surcharge, bill
  COMPARE current_units with previous_units
END IF
STOP
```

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/program.py

```
def energy_charge(units):
    if units &lt;= 100:
        return units * 2.0
    elif units &lt;= 200:
        return 100 * 2.0 + (units - 100) * 3.0
    return 100 * 2.0 + 100 * 3.0 + (units - 200) * 5.0

current = int(input("Enter current month's units: "))
previous = int(input("Enter previous month's units: "))

if current &lt; 0 or previous &lt; 0:
    print("Error: units cannot be negative.")
else:
    energy = energy_charge(current)
    surcharge = energy * 0.05
    rebate = 20 if current &lt;= 100 else 0
    total = energy + 50 + surcharge - rebate
    print(f"Energy charge: Rs.{energy:.2f}")
    print(f"Fixed charge: Rs.50.00")
    print(f"Surcharge: Rs.{surcharge:.2f}")
    print(f"Rebate: Rs.{rebate:.2f}")
    print(f"Total bill: Rs.{total:.2f}")
    if current &gt; previous:
        print(f"Usage increased by {current - previous} units.")
    elif current &lt; previous:
        print(f"Usage decreased by {previous - current} units.")
    else:
        print("Usage is unchanged.")
```

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/output.txt

Enter current month's units: 250

Enter previous month's units: 200

Energy charge: Rs.750.00

Fixed charge: Rs.50.00

Surcharge: Rs.37.50

Rebate: Rs.0.00

Total bill: Rs.837.50

Usage increased by 50 units.

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/test_cases.md

Test Case

Input / setup: Current units = 250; previous units = 200

Expected output:

```text
Enter current month's units: 250
Enter previous month's units: 200
Energy charge: Rs.750.00
Fixed charge: Rs.50.00
Surcharge: Rs.37.50
Rebate: Rs.0.00
Total bill: Rs.837.50
Usage increased by 50 units.
```

Python-Activity-Based-Learning/Case_Study_02_Progressive_Electricity_Billing/conclusion.md

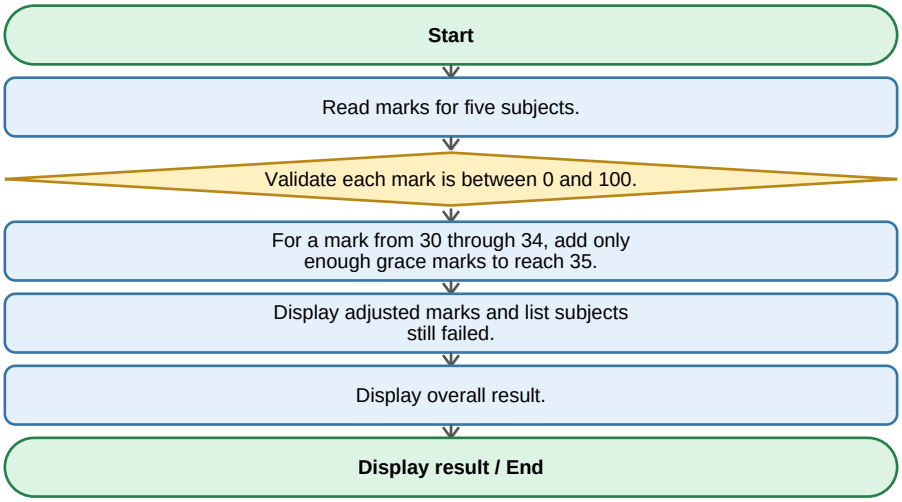
Conclusion

The program computes a bill using progressive slabs and identifies changes in monthly electricity consumption.

Case Study 11: Examination Result and Grace-Mark Validation

FLOWCHART

Examination Result and Grace-Mark Validation



Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/problem_statement.md

Examination Result and Grace-Mark Validation

Process marks in several subjects, apply a limited grace-mark policy, and generate a final result without exceeding maximum marks.

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/objectives.md

Objectives

- Validate marks.

- Apply grace marks only under the stated policy.

- Report subject marks and pass/fail status.

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/assumptions.md

Assumptions

- There are five subjects, each out of 100.
- Pass mark is 35 per subject.
- At most 5 grace marks may be added to one subject that is short by no more than 5 marks.
- Grace marks cannot make a subject exceed 100.

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/algorithm.md

Algorithm

1. Read marks for five subjects.
2. Validate each mark is between 0 and 100.
3. For a mark from 30 through 34, add only enough grace marks to reach 35.
4. Display adjusted marks and list subjects still failed.
5. Display overall result.

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/pseudocode.txt

```
START
INPUT marks for 5 subjects
FOR each mark
  IF mark is invalid: report invalid input
  ELSE IF mark is 30 to 34: raise it to 35 using grace marks
  DISPLAY adjusted marks
IF any subject is below 35: DISPLAY FAIL
ELSE: DISPLAY PASS
STOP
```

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/program.py

```
marks = []
valid = True

for i in range(5):
    mark = int(input(f"Enter marks for subject {i + 1} (0-100): "))
    if mark &lt; 0 or mark &gt; 100:
        print("Invalid marks. Each mark must be from 0 to 100.")
        valid = False
        break
    marks.append(mark)

if valid:
    adjusted = []
    grace_used = []
    for i, mark in enumerate(marks, start=1):
        if 30 &lt;= mark &lt; 35:
            needed = 35 - mark
            adjusted.append(35)
            grace_used.append((i, needed))
        else:
            adjusted.append(mark)

    print("Original marks:", marks)
    print("Final marks:", adjusted)
    if grace_used:
        print("Grace marks used:", grace_used)
    else:
        print("Grace marks used: None")

    failed = [i + 1 for i, mark in enumerate(adjusted) if mark &lt; 35]
    if failed:
        print("Result: FAIL")
        print("Failed subject numbers:", failed)
    else:
        print("Result: PASS")
    print("Total:", sum(adjusted), "/ 500")
```

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/output.txt

Enter marks for subject 1 (0-100): 78

Enter marks for subject 2 (0-100): 32

Enter marks for subject 3 (0-100): 66

Enter marks for subject 4 (0-100): 34

Enter marks for subject 5 (0-100): 90

Original marks: [78, 32, 66, 34, 90]

Final marks: [78, 35, 66, 35, 90]

Grace marks used: [(2, 3), (4, 1)]

Result: PASS

Total: 304 / 500

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/test_cases.md

Test Case

Input / setup: Marks = 78, 32, 66, 34, 90

Expected output:

```text
Enter marks for subject 1 (0-100): 78
Enter marks for subject 2 (0-100): 32
Enter marks for subject 3 (0-100): 66
Enter marks for subject 4 (0-100): 34
Enter marks for subject 5 (0-100): 90
Original marks: [78, 32, 66, 34, 90]
Final marks: [78, 35, 66, 35, 90]
Grace marks used: [(2, 3), (4, 1)]
Result: PASS
Total: 304 / 500
```

Python-Activity-Based-Learning/Case_Study_11_Examination_Result_and_Grace-Mark_Validation/conclusion.md

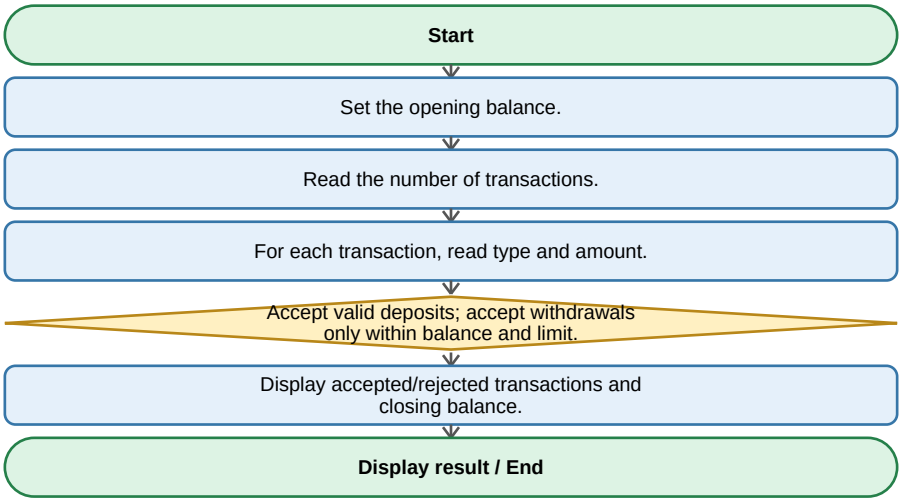
Conclusion

The program validates marks, applies the assumed grace-mark policy, and reports the final examination result.

Case Study 13: ATM Transaction Sequence Simulation

FLOWCHART

ATM Transaction Sequence Simulation



Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/problem_statement.md

ATM Transaction Sequence Simulation

Process a sequence of deposits and withdrawals, maintain the account balance, enforce a withdrawal limit, and summarize transactions.

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/objectives.md

Objectives

- Process deposits and withdrawals.

- Reject invalid amounts and withdrawals beyond available balance or the limit.

- Display a transaction summary.

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/assumptions.md

Assumptions

- Opening balance is ■5,000.
- Maximum single withdrawal is ■2,000.
- Only positive transaction amounts are accepted.

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/algorithm.md

Algorithm

1. Set the opening balance.
2. Read the number of transactions.
3. For each transaction, read type and amount.
4. Accept valid deposits; accept withdrawals only within balance and limit.
5. Display accepted/rejected transactions and closing balance.

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/pseudocode.txt

```
START
balance = 5000
INPUT number of transactions
REPEAT for each transaction
  INPUT type and amount
  IF amount &lt;= 0: reject
  ELSE IF type is DEPOSIT: add amount
  ELSE IF type is WITHDRAW: check limit and balance
  ELSE: reject unknown type
DISPLAY transaction counts and final balance
STOP
```

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/program.py

```
balance = 5000.0
accepted = 0
rejected = 0

n = int(input("How many transactions? "))
for i in range(n):
    kind = input(f"Transaction {i + 1} type (D/W): ").strip().upper()
    amount = float(input("Amount: Rs."))
    if amount &lt;= 0:
        print("Rejected: amount must be positive.")
        rejected += 1
    elif kind == "D":
        balance += amount
        print(f"Deposit accepted. Balance: Rs.{balance:.2f}")
        accepted += 1
    elif kind == "W":
        if amount &gt; 2000:
            print("Rejected: withdrawal limit is Rs.2000.")
            rejected += 1
        elif amount &gt; balance:
            print("Rejected: insufficient balance.")
            rejected += 1
        else:
            balance -= amount
            print(f"Withdrawal accepted. Balance: Rs.{balance:.2f}")
            accepted += 1
    else:
        print("Rejected: unknown transaction type.")
        rejected += 1

print(f"Accepted transactions: {accepted}")
print(f"Rejected transactions: {rejected}")
print(f"Closing balance: Rs.{balance:.2f}")
```

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/output.txt

```
How many transactions? 3
Transaction 1 type (D/W): D
Amount: Rs.1000
Deposit accepted. Balance: Rs.6000.00
Transaction 2 type (D/W): W
Amount: Rs.2500
Rejected: withdrawal limit is Rs.2000.
Transaction 3 type (D/W): W
Amount: Rs.1200
Withdrawal accepted. Balance: Rs.4800.00
Accepted transactions: 2
Rejected transactions: 1
Closing balance: Rs.4800.00
```

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/test_cases.md

Test Case

Input / setup: 3 transactions: D 1000, W 2500, W 1200

Expected output:

```text
How many transactions? 3
Transaction 1 type (D/W): D
Amount: Rs.1000
Deposit accepted. Balance: Rs.6000.00
Transaction 2 type (D/W): W
Amount: Rs.2500
Rejected: withdrawal limit is Rs.2000.
Transaction 3 type (D/W): W
Amount: Rs.1200
Withdrawal accepted. Balance: Rs.4800.00
Accepted transactions: 2
Rejected transactions: 1
Closing balance: Rs.4800.00
```

Python-Activity-Based-Learning/Case_Study_13_ATM_Transaction_Sequence_Simulation/conclusion.md

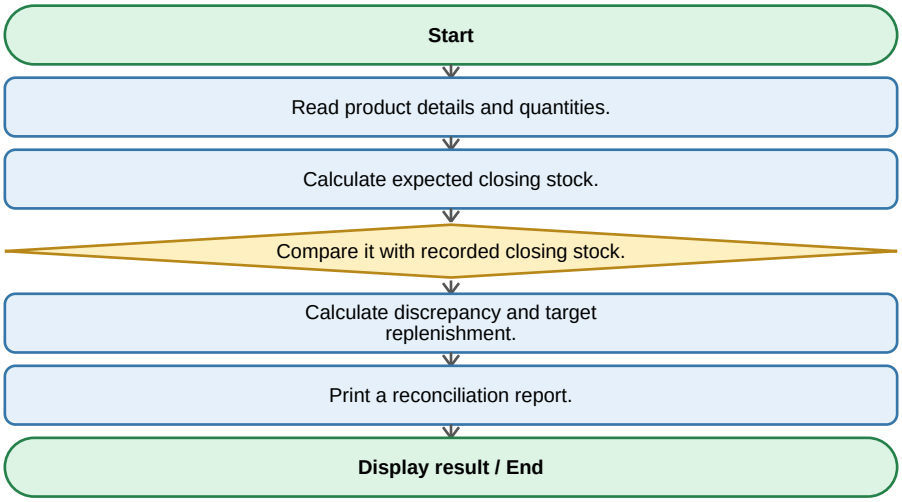
Conclusion

The simulation maintains an account balance and rejects withdrawals that violate the specified rules.

Case Study 14: Warehouse Inventory Reconciliation

FLOWCHART

Warehouse Inventory Reconciliation



Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/problem_statement.md

Warehouse Inventory Reconciliation

Compare opening stock, received quantities, dispatched quantities, and recorded closing stock to identify discrepancies and replenishment needs.

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/objectives.md

Objectives

- Calculate expected closing stock.
- Compare expected and recorded stock.
- Calculate replenishment quantities to meet a target.

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/assumptions.md

Assumptions

- The target stock for each product is 20 units.
- Expected closing stock = opening + received - dispatched.
- All quantities must be non-negative.

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/algorithm.md

Algorithm

1. Read product details and quantities.
2. Calculate expected closing stock.
3. Compare it with recorded closing stock.
4. Calculate discrepancy and target replenishment.
5. Print a reconciliation report.

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/pseudocode.txt

```
START
FOR each product
  INPUT name, opening, received, dispatched, recorded closing
  expected = opening + received - dispatched
  discrepancy = recorded closing - expected
  replenishment = max(0, 20 - recorded closing)
  DISPLAY all values
STOP
```

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/program.py

```
products = [
    ("Notebook", 50, 20, 35, 35),
    ("Pen", 100, 30, 95, 32),
    ("Folder", 25, 10, 20, 12)
]
target_stock = 20

print("Inventory Reconciliation Report")
for name, opening, received, dispatched, recorded in products:
    expected = opening + received - dispatched
    discrepancy = recorded - expected
    replenish = max(0, target_stock - recorded)
    status = "OK" if discrepancy == 0 else "DISCREPANCY"
    print(f"\nProduct: {name}")
    print(f"Expected closing stock: {expected}")
    print(f"Recorded closing stock: {recorded}")
    print(f"Difference (recorded - expected): {discrepancy}")
    print(f"Status: {status}")
    print(f"Replenishment to target {target_stock}: {replenish}")
```

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/output.txt

```
Inventory Reconciliation Report

Product: Notebook
Expected closing stock: 35
Recorded closing stock: 35
Difference (recorded - expected): 0
Status: OK
Replenishment to target 20: 0

Product: Pen
Expected closing stock: 35
Recorded closing stock: 32
Difference (recorded - expected): -3
Status: DISCREPANCY
Replenishment to target 20: 0

Product: Folder
Expected closing stock: 15
Recorded closing stock: 12
Difference (recorded - expected): -3
Status: DISCREPANCY
Replenishment to target 20: 8
```

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/test_cases.md

Test Case

Input / setup: Notebook (50, 20, 35, 35); Pen (100, 30, 95, 32); Folder (25, 10, 20, 12)

Expected output:

```text
Inventory Reconciliation Report

Product: Notebook
Expected closing stock: 35
Recorded closing stock: 35
Difference (recorded - expected): 0
Status: OK
Replenishment to target 20: 0

Product: Pen
Expected closing stock: 35
Recorded closing stock: 32
Difference (recorded - expected): -3
Status: DISCREPANCY
Replenishment to target 20: 0

Product: Folder
Expected closing stock: 15
Recorded closing stock: 12
Difference (recorded - expected): -3
Status: DISCREPANCY
Replenishment to target 20: 8
```

Python-Activity-Based-Learning/Case_Study_14_Warehouse_Inventory_Reconciliation/conclusion.md

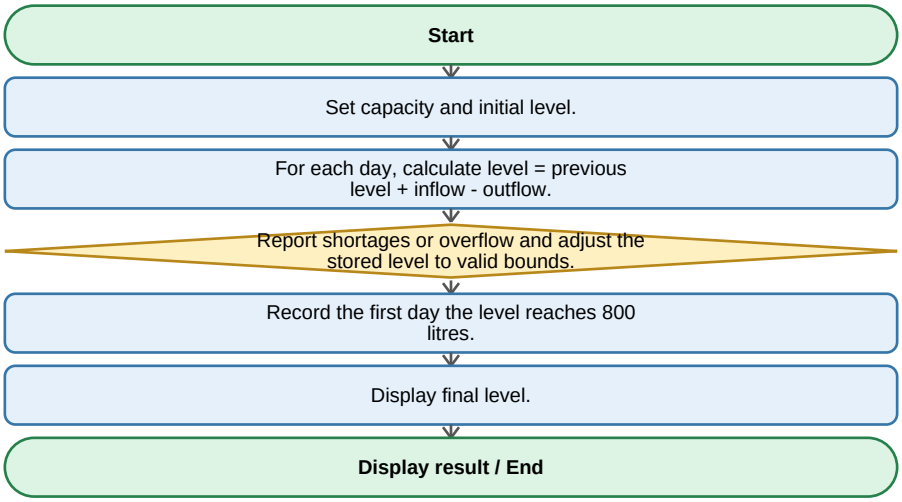
Conclusion

The program reconciles stock records, identifies differences, and calculates how much stock is needed to reach the target.

Case Study 16: Multi-Day Water Tank Simulation

FLOWCHART

Multi-Day Water Tank Simulation



Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/problem_statement.md

Multi-Day Water Tank Simulation

Track water levels over several days using inflow and outflow values, detect overflow or shortage, and find when a target level is reached.

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/objectives.md

Objectives

- Update tank level each day.

- Detect overflow and shortage.

- Identify the first day the target level is reached.

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/assumptions.md

Assumptions

- Tank capacity is 1,000 litres.
- Initial water is 400 litres.
- Daily inflow/outflow values are in litres.
- Water below zero is recorded as shortage; overflow above capacity is recorded and capped at capacity.

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/algorithm.md

Algorithm

1. Set capacity and initial level.
2. For each day, calculate level = previous level + inflow - outflow.
3. Report shortages or overflow and adjust the stored level to valid bounds.
4. Record the first day the level reaches 800 litres.
5. Display final level.

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/pseudocode.txt

```
START
capacity = 1000; level = 400; target = 800
FOR each day
  level = level + inflow - outflow
  IF level &gt; capacity: report overflow; level = capacity
  IF level &lt; 0: report shortage; level = 0
  DISPLAY level
  IF level &gt;= target and target not reached before: save day
DISPLAY first target day or not reached
STOP
```

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/program.py

```
capacity = 1000
level = 400
target = 800
daily_flow = [(300, 150), (250, 100), (100, 250), (50, 100)]

first_target_day = None
print("Day | Inflow | Outflow | End level | Status")
for day, (inflow, outflow) in enumerate(daily_flow, start=1):
    level += inflow - outflow
    status = "Normal"
    if level &gt; capacity:
        status = f"Overflow of {level - capacity} L"
        level = capacity
    elif level &lt; 0:
        status = f"Shortage of {-level} L"
        level = 0
    print(f"{day:&gt;3} | {inflow:&gt;6} | {outflow:&gt;7} | {level:&gt;9} L | {status}")
    if level &gt;= target and first_target_day is None:
        first_target_day = day

if first_target_day is None:
    print(f"Target level {target} L was not reached.")
else:
    print(f"Target level {target} L first reached on day {first_target_day}.")
print(f"Final tank level: {level} L")
```

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/output.txt

Day | Inflow | Outflow | End level | Status

1 | 300 | 150 | 550 L | Normal

2 | 250 | 100 | 700 L | Normal

3 | 100 | 250 | 550 L | Normal

4 | 50 | 100 | 500 L | Normal

Target level 800 L was not reached.

Final tank level: 500 L

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/test_cases.md

Test Case

Input / setup: Initial level 400 L; daily inflow/outflow: (300,150), (250,100), (100,250), (50,100)

Expected output:

```text
Day | Inflow | Outflow | End level | Status
1 | 300 | 150 | 550 L | Normal
2 | 250 | 100 | 700 L | Normal
3 | 100 | 250 | 550 L | Normal
4 | 50 | 100 | 500 L | Normal
Target level 800 L was not reached.
Final tank level: 500 L
```

Python-Activity-Based-Learning/Case_Study_16_Multi-Day_Water_Tank_Simulation/conclusion.md

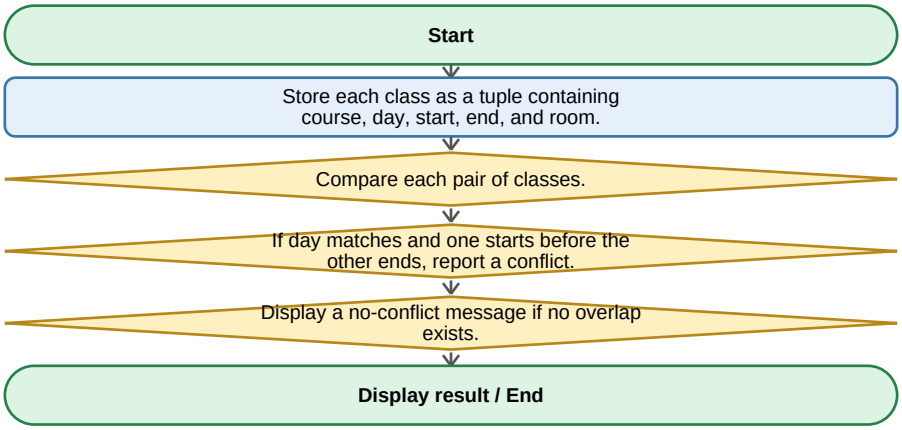
Conclusion

The simulation updates water levels over time and detects whether a target level, overflow, or shortage occurs.

Case Study 23: Student Timetable Conflict Detection

FLOWCHART

Student Timetable Conflict Detection



Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/problem_statement.md

Student Timetable Conflict Detection

Check a proposed timetable for overlapping class periods and identify conflicts between courses.

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/objectives.md

Objectives

- Represent course schedules using tuples.
- Detect time overlaps.
- Report conflicting course pairs.

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/assumptions.md

Assumptions

- Times use 24-hour HHMM integers.

- A class interval is treated as [start, end).

- Two classes conflict only when they are on the same day and their intervals overlap.

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/algorithm.md

Algorithm

1. Store each class as a tuple containing course, day, start, end, and room.
2. Compare each pair of classes.
3. If day matches and one starts before the other ends, report a conflict.
4. Display a no-conflict message if no overlap exists.

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/pseudocode.txt

```
START
FOR i from 0 to number_of_classes - 1
  FOR j from i + 1 to number_of_classes - 1
    IF day[i] = day[j] AND start[i] &lt; end[j] AND start[j] &lt; end[i]
      DISPLAY conflicting courses
IF no conflict: DISPLAY no conflicts
STOP
```

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/program.py

```
classes = [
    ("Python", "Monday", 900, 1000, "Lab 1"),
    ("Mathematics", "Monday", 930, 1030, "Room 2"),
    ("Physics", "Monday", 1100, 1200, "Room 3"),
    ("English", "Tuesday", 900, 1000, "Room 4")
]

conflicts = []
for i in range(len(classes)):
    course1, day1, start1, end1, room1 = classes[i]
    for j in range(i + 1, len(classes)):
        course2, day2, start2, end2, room2 = classes[j]
        if day1 == day2 and start1 &lt; end2 and start2 &lt; end1:
            conflicts.append((course1, course2, day1))

if conflicts:
    print("Timetable conflicts found:")
    for course1, course2, day in conflicts:
        print(f"{course1} conflicts with {course2} on {day}.")
else:
    print("No timetable conflicts found.")
```

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/output.txt

```
Timetable conflicts found:
Python conflicts with Mathematics on Monday.
```

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/test_cases.md

Test Case

Input / setup: Python Monday 09:00-10:00; Mathematics Monday 09:30-10:30; Physics Monday 11:00-12:00; English Tuesday 09:00-10:00

Expected output:

```text
Timetable conflicts found:

Python conflicts with Mathematics on Monday.
```

Python-Activity-Based-Learning/Case_Study_23_Student_Timetable_Conflict_Detection/conclusion.md

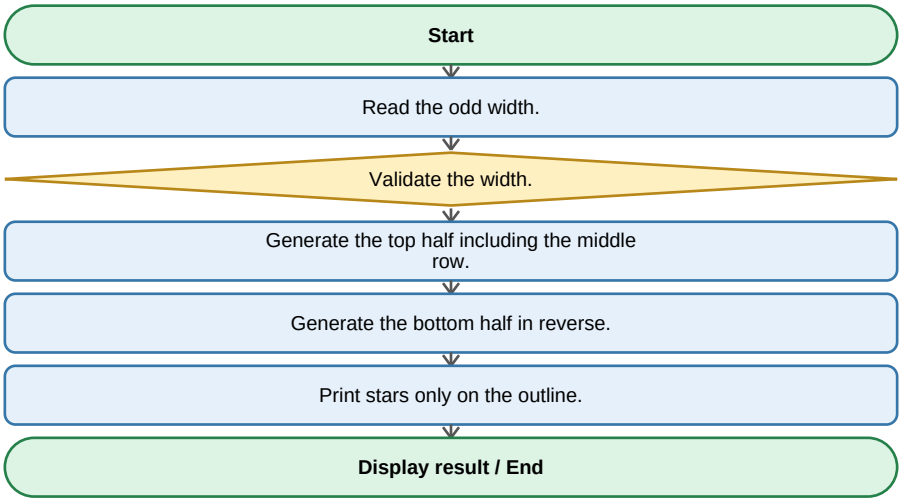
Conclusion

The program uses tuples and nested loops to identify overlapping classes on the same day.

Case Study 27: Advanced Numeric Pattern Generator

FLOWCHART

Advanced Numeric Pattern Generator



Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/problem_statement.md

Advanced Numeric Pattern Generator

Generate a hollow diamond pattern for a user-specified odd width and reject invalid dimensions.

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/objectives.md

Objectives

- Use nested loops to generate a symmetric pattern.
- Print only the boundary stars.
- Validate the requested size.

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/assumptions.md

Assumptions

- The size must be an odd integer from 3 through 19.

- The input size represents the widest row.

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/algorithm.md

Algorithm

1. Read the odd width.
2. Validate the width.
3. Generate the top half including the middle row.
4. Generate the bottom half in reverse.
5. Print stars only on the outline.

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/pseudocode.txt

```
START
INPUT width
IF width is even or outside 3..19: DISPLAY error
ELSE
  n = width // 2
  FOR row from 0 to n
    print leading spaces and hollow row
  FOR row from n-1 down to 0
    print leading spaces and hollow row
STOP
```

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/program.py

```
width = int(input("Enter an odd diamond width (3-19): "))

if width &lt; 3 or width &gt; 19 or width % 2 == 0:
    print("Invalid width. Enter an odd number from 3 to 19.")
else:
    middle = width // 2
    for row in list(range(middle + 1)) + list(range(middle - 1, -1, -1)):
        spaces = middle - row
        if row == 0:
            print(" " * spaces + "*")
        else:
            inside = 2 * row - 1
            print(" " * spaces + "*" + " " * inside + "*")
```

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/output.txt

```
Enter an odd diamond width (3-19): 7
   *
  * *
 * *
* *
 * *
  * *
   *
```

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/test_cases.md

Test Case

Input / setup: Width = 7

Expected output:

```text
Enter an odd diamond width (3-19): 7
*
* *
* *
* *
* *
* *
*
```

Python-Activity-Based-Learning/Case_Study_27_Advanced_Numeric_Pattern_Generator/conclusion.md

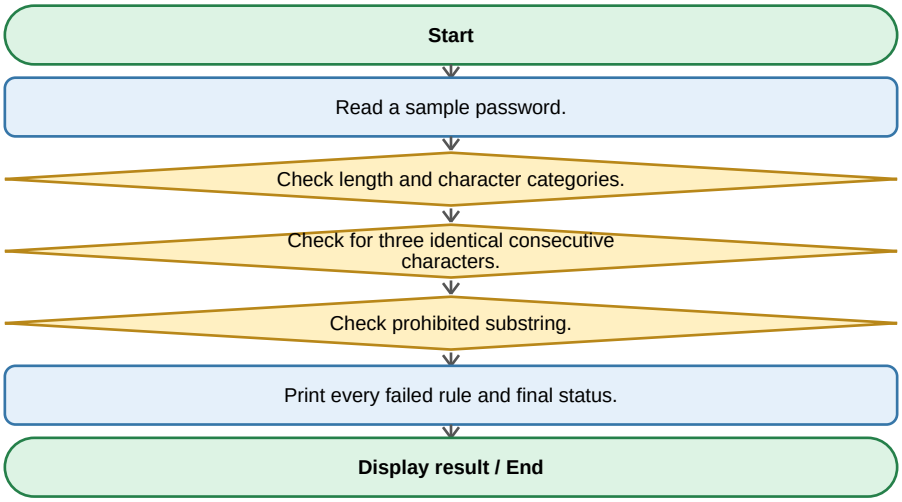
Conclusion

The program generates a symmetric hollow diamond using loops and validates the input dimension.

Case Study 29: Password Policy Validation Engine

FLOWCHART

Password Policy Validation Engine



Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/problem_statement.md

Password Policy Validation Engine

Evaluate a sample password against length, character-category, repeated-character, and prohibited-substring rules.

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/objectives.md

Objectives

- Check several password rules.
- Explain which conditions fail.
- Display a final valid/invalid status.

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/assumptions.md

Assumptions

- Minimum length is 8.
- A password must contain uppercase, lowercase, digit, and special character.
- Three identical consecutive characters are prohibited.
- The substring 'password' is prohibited without regard to case.

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/algorithm.md

Algorithm

1. Read a sample password.
2. Check length and character categories.
3. Check for three identical consecutive characters.
4. Check prohibited substring.
5. Print every failed rule and final status.

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/pseudocode.txt

```
START
INPUT password
Create list of failed rules
CHECK length &gt;= 8
CHECK uppercase, lowercase, digit, special character
CHECK no 3 identical consecutive characters
CHECK 'password' is not in lowercase password
IF no failures: DISPLAY valid
ELSE DISPLAY failed rules
STOP
```

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/program.py

```
password = input("Enter a sample password to check: ")
failures = []

if len(password) &lt; 8:
    failures.append("Must contain at least 8 characters.")
if not any(ch.isupper() for ch in password):
    failures.append("Must contain an uppercase letter.")
if not any(ch.islower() for ch in password):
    failures.append("Must contain a lowercase letter.")
if not any(ch.isdigit() for ch in password):
    failures.append("Must contain a digit.")
if not any(not ch.isalnum() for ch in password):
    failures.append("Must contain a special character.")
if any(password[i] == password[i+1] == password[i+2]
       for i in range(len(password) - 2)):
    failures.append("Must not contain 3 identical consecutive characters.")
if "password" in password.lower():
    failures.append("Must not contain the word 'password'.")

if failures:
    print("Password: INVALID")
    for item in failures:
        print("-", item)
else:
    print("Password: VALID")
print("Note: This is a local demonstration; do not enter a real password.")
```

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/output.txt

```
Enter a sample password to check: Pass123!

Password: VALID

Note: This is a local demonstration; do not enter a real password.
```

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/test_cases.md

Test Case

Input / setup: Sample input = Pass123!

Expected output:

```text
Enter a sample password to check: Pass123!
Password: VALID
Note: This is a local demonstration; do not enter a real password.
```

Python-Activity-Based-Learning/Case_Study_29_Password_Policy_Validation_Engine/conclusion.md

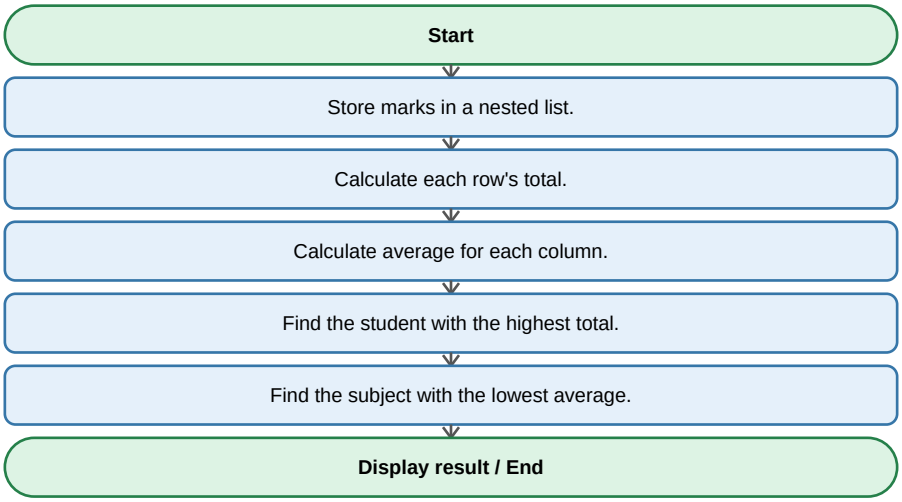
Conclusion

The program checks multiple string-based password rules and explains any failed conditions. Only use fictional sample passwords for testing.

Case Study 42: Student Marks Matrix Analysis

FLOWCHART

Student Marks Matrix Analysis



Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/problem_statement.md

Student Marks Matrix Analysis

Analyze a nested list of marks to calculate each student's total and each subject's average, then identify the top student and weakest-average subject.

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/objectives.md

Objectives

- Calculate row-wise totals.

- Calculate column-wise averages.

- Identify top performer and lowest-average subject.

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/assumptions.md

Assumptions

- Rows represent students and columns represent subjects.

- All marks are out of 100.

- The sample data contains valid marks.

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/algorithm.md

Algorithm

1. Store marks in a nested list.
2. Calculate each row's total.
3. Calculate average for each column.
4. Find the student with the highest total.
5. Find the subject with the lowest average.

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/pseudocode.txt

```
START
STORE marks matrix
FOR each student row: calculate total
FOR each subject column: calculate average
FIND maximum student total
FIND minimum subject average
DISPLAY totals, averages, top student, weakest subject
STOP
```

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/program.py

```
students = ["Asha", "Bharat", "Charan"]
subjects = ["Python", "Maths", "Physics"]
marks = [
    [88, 76, 91],
    [72, 85, 70],
    [95, 89, 84]
]

totals = [sum(row) for row in marks]
averages = []
for col in range(len(subjects)):
    subject_total = 0
    for row in marks:
        subject_total += row[col]
    averages.append(subject_total / len(marks))

top_index = totals.index(max(totals))
weakest_index = averages.index(min(averages))

print("Student totals:")
for i, name in enumerate(students):
    print(f"{name}: {totals[i]}/300")
print("\nSubject averages:")
for i, subject in enumerate(subjects):
    print(f"{subject}: {averages[i]:.2f}")
print(f"Top performer: {students[top_index]}")
print(f"Weakest-average subject: {subjects[weakest_index]}")
```

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/output.txt

```
Student totals:
Asha: 255/300
Bharat: 227/300
Charan: 268/300

Subject averages:
Python: 85.00
Maths: 83.33
Physics: 81.67
Top performer: Charan
Weakest-average subject: Physics
```

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/test_cases.md

Test Case

Input / setup: Asha [88,76,91]; Bharat [72,85,70]; Charan [95,89,84]

Expected output:

```text
Student totals:
Asha: 255/300
Bharat: 227/300
Charan: 268/300

Subject averages:
Python: 85.00
Maths: 83.33
Physics: 81.67
Top performer: Charan
Weakest-average subject: Physics
```

Python-Activity-Based-Learning/Case_Study_42_Student_Marks_Matrix_Analysis/conclusion.md

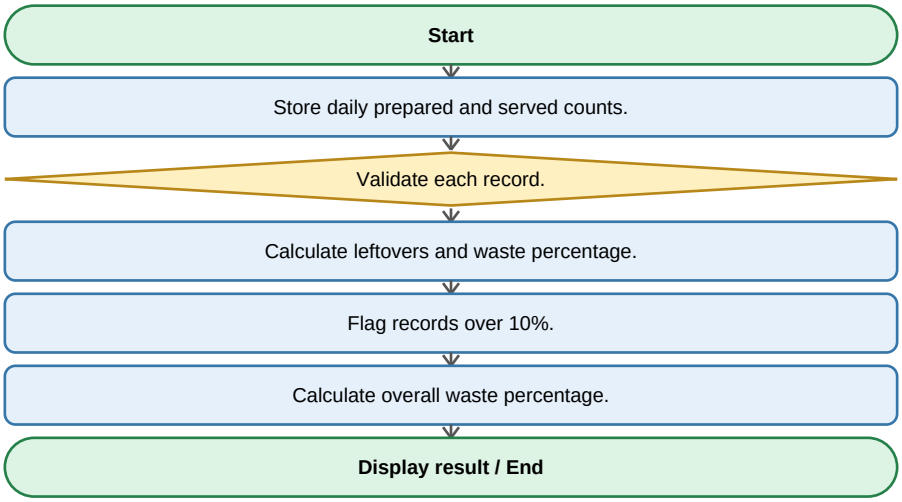
Conclusion

The program uses nested lists and loops to summarize student performance by student and by subject.

Case Study 53: Food Waste Monitoring System

FLOWCHART

Food Waste Monitoring System



Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/problem_statement.md

Food Waste Monitoring System

Compare meals prepared, meals served, and leftovers across several days; calculate waste percentages and flag days that exceed an acceptable limit.

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/objectives.md

Objectives

- Calculate leftovers.
- Calculate waste percentage for each day.
- Identify days exceeding the waste threshold.

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/assumptions.md

Assumptions

- Waste percentage = leftovers / prepared meals × 100.
- Acceptable waste limit is 10%.
- Prepared meals must be greater than zero and served meals cannot exceed prepared meals.

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/algorithm.md

Algorithm

1. Store daily prepared and served counts.
2. Validate each record.
3. Calculate leftovers and waste percentage.
4. Flag records over 10%.
5. Calculate overall waste percentage.

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/pseudocode.txt

```
START
STORE daily prepared and served values
FOR each day
  IF prepared &lt;= 0 OR served &lt; 0 OR served &gt; prepared: report invalid record
  ELSE leftovers = prepared - served
       waste_pct = leftovers / prepared * 100
       IF waste_pct &gt; 10: flag day
CALCULATE overall waste percentage
DISPLAY report
STOP
```

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/program.py

```
days = [
    ("Monday", 200, 185),
    ("Tuesday", 220, 190),
    ("Wednesday", 180, 175),
    ("Thursday", 210, 180)
]
limit = 10
total_prepared = 0
total_leftovers = 0

print("Food Waste Report (limit: 10%)")
for day, prepared, served in days:
    if prepared &lt;= 0 or served &lt; 0 or served &gt; prepared:
        print(f"{day}: Invalid data")
        continue
    leftovers = prepared - served
    percentage = leftovers / prepared * 100
    total_prepared += prepared
    total_leftovers += leftovers
    status = "EXCEEDS LIMIT" if percentage &gt; limit else "Within limit"
    print(f"{day}: prepared={prepared}, served={served}, leftovers={leftovers}, waste={percentage:.2f}% -
 {status}")

overall = total_leftovers / total_prepared * 100
print(f"Overall leftovers: {total_leftovers}")
print(f"Overall waste percentage: {overall:.2f}%")
```

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/output.txt

Food Waste Report (limit: 10%)

Monday: prepared=200, served=185, leftovers=15, waste=7.50% - Within limit

Tuesday: prepared=220, served=190, leftovers=30, waste=13.64% - EXCEEDS LIMIT

Wednesday: prepared=180, served=175, leftovers=5, waste=2.78% - Within limit

Thursday: prepared=210, served=180, leftovers=30, waste=14.29% - EXCEEDS LIMIT

Overall leftovers: 80

Overall waste percentage: 9.88%

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/test_cases.md

Test Case

Input / setup: Monday 200/185; Tuesday 220/190; Wednesday 180/175; Thursday 210/180

Expected output:

```text
Food Waste Report (limit: 10%)
Monday: prepared=200, served=185, leftovers=15, waste=7.50% - Within limit
Tuesday: prepared=220, served=190, leftovers=30, waste=13.64% - EXCEEDS LIMIT
Wednesday: prepared=180, served=175, leftovers=5, waste=2.78% - Within limit
Thursday: prepared=210, served=180, leftovers=30, waste=14.29% - EXCEEDS LIMIT
Overall leftovers: 80
Overall waste percentage: 9.88%
```

Python-Activity-Based-Learning/Case_Study_53_Food_Waste_Monitoring_System/conclusion.md

Conclusion

The program calculates daily and overall food waste and highlights days that exceed the allowed threshold.

Python Activity-Based Learning (ABL)

10 Selected Case Studies — Solutions and Sample Outputs

Student name: Lingam Sasank Venkat

Roll number: 26000938

Branch: CSM Section: 2

Important note

These are 10 selected problems from the supplied faculty list. Tariff values, grace-mark rules, limits, and thresholds are stated assumptions for demonstration and should be adjusted if faculty specifies different rules. Outputs are sample runs. Run each program yourself and add your own screenshots and flowchart images before submission.

Selected case studies

Progressive Electricity Billing

Examination Result and Grace-Mark Validation

ATM Transaction Sequence Simulation

Warehouse Inventory Reconciliation

Multi-Day Water Tank Simulation

Student Timetable Conflict Detection

Advanced Numeric Pattern Generator

Password Policy Validation Engine

Student Marks Matrix Analysis

Food Waste Monitoring System

Progressive Electricity Billing

Problem statement

Calculate a household's electricity bill using progressive consumption slabs, a fixed charge, surcharge, and rebate.

Objectives

• Calculate energy charges across multiple slabs.
• Apply fixed charges, surcharge, and rebate.
• Compare the current bill with the previous month's bill.

Assumptions

• First 100 units cost ■2.00 per unit.
• Next 100 units cost ■3.00 per unit.
• Units above 200 cost ■5.00 per unit.
• Fixed charge is ■50; surcharge is 5% of energy charges; rebate is ■20 when consumption is at most 100 units.

Algorithm

1. Read current and previous consumption.
2. Validate that both readings are non-negative.
3. Calculate progressive energy charges by slab.
4. Add fixed charge and surcharge; apply the eligible rebate.
5. Display the bill and compare it with the previous month.

Pseudocode

```
START
INPUT current_units, previous_units
IF either value is negative: DISPLAY error
ELSE
  CALCULATE energy charge by progressive slabs
  surcharge = energy charge * 5 / 100
  bill = energy charge + 50 + surcharge
  IF current_units <= 100: bill = bill - 20
  DISPLAY energy charge, surcharge, bill
  COMPARE current_units with previous_units
END IF
STOP
```

Python source code

```
def energy_charge(units):
    if units <= 100:
        return units * 2.0
    elif units <= 200:
        return 100 * 2.0 + (units - 100) * 3.0
    return 100 * 2.0 + 100 * 3.0 + (units - 200) * 5.0

current = int(input("Enter current month's units: "))
previous = int(input("Enter previous month's units: "))

if current < 0 or previous < 0:
    print("Error: units cannot be negative.")
else:
    energy = energy_charge(current)
    surcharge = energy * 0.05
    rebate = 20 if current <= 100 else 0
    total = energy + 50 + surcharge - rebate
    print(f"Energy charge: Rs.{energy:.2f}")
    print(f"Fixed charge: Rs.50.00")
    print(f"Surcharge: Rs.{surcharge:.2f}")
    print(f"Rebate: Rs.{rebate:.2f}")
    print(f"Total bill: Rs.{total:.2f}")
    if current > previous:
        print(f"Usage increased by {current - previous} units.")
    elif current < previous:
        print(f"Usage decreased by {previous - current} units.")
    else:
        print("Usage is unchanged.")
```

Test case

Current units = 250; previous units = 200

Sample output

```
Enter current month's units: 250
Enter previous month's units: 200
Energy charge: Rs.750.00
Fixed charge: Rs.50.00
Surcharge: Rs.37.50
```

```
Rebate: Rs.0.00
Total bill: Rs.837.50
Usage increased by 50 units.
```

Conclusion

The program computes a bill using progressive slabs and identifies changes in monthly electricity consumption.

Examination Result and Grace-Mark Validation

Problem statement

Process marks in several subjects, apply a limited grace-mark policy, and generate a final result without exceeding maximum marks.

Objectives

• Validate marks.
• Apply grace marks only under the stated policy.
• Report subject marks and pass/fail status.

Assumptions

• There are five subjects, each out of 100.
• Pass mark is 35 per subject.
• At most 5 grace marks may be added to one subject that is short by no more than 5 marks.
• Grace marks cannot make a subject exceed 100.

Algorithm

1. Read marks for five subjects.
2. Validate each mark is between 0 and 100.
3. For a mark from 30 through 34, add only enough grace marks to reach 35.
4. Display adjusted marks and list subjects still failed.
5. Display overall result.

Pseudocode

```
START
INPUT marks for 5 subjects
FOR each mark
  IF mark is invalid: report invalid input
  ELSE IF mark is 30 to 34: raise it to 35 using grace marks
  DISPLAY adjusted marks
IF any subject is below 35: DISPLAY FAIL
ELSE: DISPLAY PASS
STOP
```

Python source code

```
marks = []
valid = True

for i in range(5):
    mark = int(input(f"Enter marks for subject {i + 1} (0-100): "))
    if mark < 0 or mark > 100:
        print("Invalid marks. Each mark must be from 0 to 100.")
        valid = False
        break
    marks.append(mark)

if valid:
    adjusted = []
    grace_used = []
    for i, mark in enumerate(marks, start=1):
        if 30 <= mark < 35:
            needed = 35 - mark
            adjusted.append(35)
            grace_used.append((i, needed))
        else:
            adjusted.append(mark)

    print("Original marks:", marks)
    print("Final marks:", adjusted)
    if grace_used:
        print("Grace marks used:", grace_used)
    else:
        print("Grace marks used: None")

    failed = [i + 1 for i, mark in enumerate(adjusted) if mark < 35]
    if failed:
        print("Result: FAIL")
        print("Failed subject numbers:", failed)
    else:
        print("Result: PASS")
    print("Total:", sum(adjusted), "/ 500")
```

Test case

Marks = 78, 32, 66, 34, 90

Sample output

```
Enter marks for subject 1 (0-100): 78
Enter marks for subject 2 (0-100): 32
Enter marks for subject 3 (0-100): 66
Enter marks for subject 4 (0-100): 34
Enter marks for subject 5 (0-100): 90
Original marks: [78, 32, 66, 34, 90]
Final marks: [78, 35, 66, 35, 90]
Grace marks used: [(2, 3), (4, 1)]
Result: PASS
Total: 304 / 500
```

Conclusion

The program validates marks, applies the assumed grace-mark policy, and reports the final examination result.

ATM Transaction Sequence Simulation

Problem statement

Process a sequence of deposits and withdrawals, maintain the account balance, enforce a withdrawal limit, and summarize transactions.

Objectives

• Process deposits and withdrawals.
• Reject invalid amounts and withdrawals beyond available balance or the limit.
• Display a transaction summary.

Assumptions

• Opening balance is ■5,000.
• Maximum single withdrawal is ■2,000.
• Only positive transaction amounts are accepted.

Algorithm

1. Set the opening balance.
2. Read the number of transactions.
3. For each transaction, read type and amount.
4. Accept valid deposits; accept withdrawals only within balance and limit.
5. Display accepted/rejected transactions and closing balance.

Pseudocode

```
START
balance = 5000
INPUT number of transactions
REPEAT for each transaction
  INPUT type and amount
  IF amount <= 0: reject
  ELSE IF type is DEPOSIT: add amount
  ELSE IF type is WITHDRAW: check limit and balance
  ELSE: reject unknown type
DISPLAY transaction counts and final balance
STOP
```

Python source code

```
balance = 5000.0
accepted = 0
rejected = 0

n = int(input("How many transactions? "))
for i in range(n):
    kind = input(f"Transaction {i + 1} type (D/W): ").strip().upper()
    amount = float(input("Amount: Rs."))
    if amount <= 0:
        print("Rejected: amount must be positive.")
        rejected += 1
    elif kind == "D":
        balance += amount
        print(f"Deposit accepted. Balance: Rs.{balance:.2f}")
        accepted += 1
    elif kind == "W":
        if amount > 2000:
            print("Rejected: withdrawal limit is Rs.2000.")
            rejected += 1
        elif amount > balance:
            print("Rejected: insufficient balance.")
            rejected += 1
        else:
            balance -= amount
            print(f"Withdrawal accepted. Balance: Rs.{balance:.2f}")
            accepted += 1
    else:
        print("Rejected: unknown transaction type.")
        rejected += 1

print(f"Accepted transactions: {accepted}")
print(f"Rejected transactions: {rejected}")
print(f"Closing balance: Rs.{balance:.2f}")
```

Test case

3 transactions: D 1000, W 2500, W 1200

Sample output

```
How many transactions? 3
```

```
Transaction 1 type (D/W): D
Amount: Rs.1000
Deposit accepted. Balance: Rs.6000.00
Transaction 2 type (D/W): W
Amount: Rs.2500
Rejected: withdrawal limit is Rs.2000.
Transaction 3 type (D/W): W
Amount: Rs.1200
Withdrawal accepted. Balance: Rs.4800.00
Accepted transactions: 2
Rejected transactions: 1
Closing balance: Rs.4800.00
```

Conclusion

The simulation maintains an account balance and rejects withdrawals that violate the specified rules.

Warehouse Inventory Reconciliation

Problem statement

Compare opening stock, received quantities, dispatched quantities, and recorded closing stock to identify discrepancies and replenishment needs.

Objectives

• Calculate expected closing stock.
• Compare expected and recorded stock.
• Calculate replenishment quantities to meet a target.

Assumptions

• The target stock for each product is 20 units.
• Expected closing stock = opening + received - dispatched.
• All quantities must be non-negative.

Algorithm

1. Read product details and quantities.
2. Calculate expected closing stock.
3. Compare it with recorded closing stock.
4. Calculate discrepancy and target replenishment.
5. Print a reconciliation report.

Pseudocode

```
START
FOR each product
  INPUT name, opening, received, dispatched, recorded closing
  expected = opening + received - dispatched
  discrepancy = recorded closing - expected
  replenishment = max(0, 20 - recorded closing)
  DISPLAY all values
STOP
```

Python source code

```
products = [
    ("Notebook", 50, 20, 35, 35),
    ("Pen", 100, 30, 95, 32),
    ("Folder", 25, 10, 20, 12)
]
target_stock = 20

print("Inventory Reconciliation Report")
for name, opening, received, dispatched, recorded in products:
    expected = opening + received - dispatched
    discrepancy = recorded - expected
    replenish = max(0, target_stock - recorded)
    status = "OK" if discrepancy == 0 else "DISCREPANCY"
    print(f"\nProduct: {name}")
    print(f"Expected closing stock: {expected}")
    print(f"Recorded closing stock: {recorded}")
    print(f"Difference (recorded - expected): {discrepancy}")
    print(f"Status: {status}")
    print(f"Replenishment to target {target_stock}: {replenish}")
```

Test case

Notebook (50, 20, 35, 35); Pen (100, 30, 95, 32); Folder (25, 10, 20, 12)

Sample output

```
Inventory Reconciliation Report

Product: Notebook
Expected closing stock: 35
Recorded closing stock: 35
Difference (recorded - expected): 0
Status: OK
Replenishment to target 20: 0

Product: Pen
Expected closing stock: 35
Recorded closing stock: 32
Difference (recorded - expected): -3
Status: DISCREPANCY
Replenishment to target 20: 0

Product: Folder
Expected closing stock: 15
```

```
Recorded closing stock: 12
Difference (recorded - expected): -3
Status: DISCREPANCY
Replenishment to target 20: 8
```

Conclusion

The program reconciles stock records, identifies differences, and calculates how much stock is needed to reach the target.

Multi-Day Water Tank Simulation

Problem statement

Track water levels over several days using inflow and outflow values, detect overflow or shortage, and find when a target level is reached.

Objectives

• Update tank level each day.
• Detect overflow and shortage.
• Identify the first day the target level is reached.

Assumptions

• Tank capacity is 1,000 litres.
• Initial water is 400 litres.
• Daily inflow/outflow values are in litres.
• Water below zero is recorded as shortage; overflow above capacity is recorded and capped at capacity.

Algorithm

1. Set capacity and initial level.
2. For each day, calculate level = previous level + inflow - outflow.
3. Report shortages or overflow and adjust the stored level to valid bounds.
4. Record the first day the level reaches 800 litres.
5. Display final level.

Pseudocode

```
START
capacity = 1000; level = 400; target = 800
FOR each day
  level = level + inflow - outflow
  IF level > capacity: report overflow; level = capacity
  IF level < 0: report shortage; level = 0
  DISPLAY level
  IF level >= target and target not reached before: save day
DISPLAY first target day or not reached
STOP
```

Python source code

```
capacity = 1000
level = 400
target = 800
daily_flow = [(300, 150), (250, 100), (100, 250), (50, 100)]

first_target_day = None
print("Day | Inflow | Outflow | End level | Status")
for day, (inflow, outflow) in enumerate(daily_flow, start=1):
    level += inflow - outflow
    status = "Normal"
    if level > capacity:
        status = f"Overflow of {level - capacity} L"
        level = capacity
    elif level < 0:
        status = f"Shortage of {-level} L"
        level = 0
    print(f"{day:>3} | {inflow:>6} | {outflow:>7} | {level:>9} L | {status}")
    if level >= target and first_target_day is None:
        first_target_day = day

if first_target_day is None:
    print(f"Target level {target} L was not reached.")
else:
    print(f"Target level {target} L first reached on day {first_target_day}.")
print(f"Final tank level: {level} L")
```

Test case

Initial level 400 L; daily inflow/outflow: (300,150), (250,100), (100,250), (50,100)

Sample output

```
Day | Inflow | Outflow | End level | Status
  1 |    300 |     150 |       550 L | Normal
  2 |    250 |     100 |       700 L | Normal
  3 |    100 |     250 |       550 L | Normal
  4 |     50 |     100 |       500 L | Normal
Target level 800 L was not reached.
Final tank level: 500 L
```

Conclusion

The simulation updates water levels over time and detects whether a target level, overflow, or shortage occurs.

Student Timetable Conflict Detection

Problem statement

Check a proposed timetable for overlapping class periods and identify conflicts between courses.

Objectives

• Represent course schedules using tuples.
• Detect time overlaps.
• Report conflicting course pairs.

Assumptions

• Times use 24-hour HHMM integers.
• A class interval is treated as [start, end).
• Two classes conflict only when they are on the same day and their intervals overlap.

Algorithm

1. Store each class as a tuple containing course, day, start, end, and room.
2. Compare each pair of classes.
3. If day matches and one starts before the other ends, report a conflict.
4. Display a no-conflict message if no overlap exists.

Pseudocode

```
START
FOR i from 0 to number_of_classes - 1
  FOR j from i + 1 to number_of_classes - 1
    IF day[i] = day[j] AND start[i] < end[j] AND start[j] < end[i]
      DISPLAY conflicting courses
IF no conflict: DISPLAY no conflicts
STOP
```

Python source code

```
classes = [
    ("Python", "Monday", 900, 1000, "Lab 1"),
    ("Mathematics", "Monday", 930, 1030, "Room 2"),
    ("Physics", "Monday", 1100, 1200, "Room 3"),
    ("English", "Tuesday", 900, 1000, "Room 4")
]

conflicts = []
for i in range(len(classes)):
    course1, day1, start1, end1, room1 = classes[i]
    for j in range(i + 1, len(classes)):
        course2, day2, start2, end2, room2 = classes[j]
        if day1 == day2 and start1 < end2 and start2 < end1:
            conflicts.append((course1, course2, day1))

if conflicts:
    print("Timetable conflicts found:")
    for course1, course2, day in conflicts:
        print(f"{course1} conflicts with {course2} on {day}.")
else:
    print("No timetable conflicts found.")
```

Test case

Python Monday 09:00-10:00; Mathematics Monday 09:30-10:30; Physics Monday 11:00-12:00; English Tuesday 09:00-10:00

Sample output

```
Timetable conflicts found:
Python conflicts with Mathematics on Monday.
```

Conclusion

The program uses tuples and nested loops to identify overlapping classes on the same day.

Advanced Numeric Pattern Generator

Problem statement

Generate a hollow diamond pattern for a user-specified odd width and reject invalid dimensions.

Objectives

• Use nested loops to generate a symmetric pattern.
• Print only the boundary stars.
• Validate the requested size.

Assumptions

• The size must be an odd integer from 3 through 19.
• The input size represents the widest row.

Algorithm

1. Read the odd width.
2. Validate the width.
3. Generate the top half including the middle row.
4. Generate the bottom half in reverse.
5. Print stars only on the outline.

Pseudocode

```
START
INPUT width
IF width is even or outside 3..19: DISPLAY error
ELSE
  n = width // 2
  FOR row from 0 to n
    print leading spaces and hollow row
  FOR row from n-1 down to 0
    print leading spaces and hollow row
STOP
```

Python source code

```
width = int(input("Enter an odd diamond width (3-19): "))

if width < 3 or width > 19 or width % 2 == 0:
    print("Invalid width. Enter an odd number from 3 to 19.")
else:
    middle = width // 2
    for row in list(range(middle + 1)) + list(range(middle - 1, -1, -1)):
        spaces = middle - row
        if row == 0:
            print(" " * spaces + "*")
        else:
            inside = 2 * row - 1
            print(" " * spaces + "*" + " " * inside + "*")
```

Test case

Width = 7

Sample output

```
Enter an odd diamond width (3-19): 7
   *
  * *
 *   *
*     *
 *   *
  * *
   *
```

Conclusion

The program generates a symmetric hollow diamond using loops and validates the input dimension.

Password Policy Validation Engine

Problem statement

Evaluate a sample password against length, character-category, repeated-character, and prohibited-substring rules.

Objectives

• Check several password rules.
• Explain which conditions fail.
• Display a final valid/invalid status.

Assumptions

• Minimum length is 8.
• A password must contain uppercase, lowercase, digit, and special character.
• Three identical consecutive characters are prohibited.
• The substring 'password' is prohibited without regard to case.

Algorithm

1. Read a sample password.
2. Check length and character categories.
3. Check for three identical consecutive characters.
4. Check prohibited substring.
5. Print every failed rule and final status.

Pseudocode

```
START
INPUT password
Create list of failed rules
CHECK length >= 8
CHECK uppercase, lowercase, digit, special character
CHECK no 3 identical consecutive characters
CHECK 'password' is not in lowercase password
IF no failures: DISPLAY valid
ELSE DISPLAY failed rules
STOP
```

Python source code

```
password = input("Enter a sample password to check: ")
failures = []

if len(password) < 8:
    failures.append("Must contain at least 8 characters.")
if not any(ch.isupper() for ch in password):
    failures.append("Must contain an uppercase letter.")
if not any(ch.islower() for ch in password):
    failures.append("Must contain a lowercase letter.")
if not any(ch.isdigit() for ch in password):
    failures.append("Must contain a digit.")
if not any(not ch.isalnum() for ch in password):
    failures.append("Must contain a special character.")
if any(password[i] == password[i+1] == password[i+2]
       for i in range(len(password) - 2)):
    failures.append("Must not contain 3 identical consecutive characters.")
if "password" in password.lower():
    failures.append("Must not contain the word 'password'.")

if failures:
    print("Password: INVALID")
    for item in failures:
        print("-", item)
else:
    print("Password: VALID")
print("Note: This is a local demonstration; do not enter a real password.")
```

Test case

Sample input = Pass123!

Sample output

```
Enter a sample password to check: Pass123!
Password: VALID
Note: This is a local demonstration; do not enter a real password.
```

Conclusion

The program checks multiple string-based password rules and explains any failed conditions. Only use fictional sample passwords for testing.

Student Marks Matrix Analysis

Problem statement

Analyze a nested list of marks to calculate each student's total and each subject's average, then identify the top student and weakest-average subject.

Objectives

• Calculate row-wise totals.
• Calculate column-wise averages.
• Identify top performer and lowest-average subject.

Assumptions

• Rows represent students and columns represent subjects.
• All marks are out of 100.
• The sample data contains valid marks.

Algorithm

1. Store marks in a nested list.
2. Calculate each row's total.
3. Calculate average for each column.
4. Find the student with the highest total.
5. Find the subject with the lowest average.

Pseudocode

```
START
STORE marks matrix
FOR each student row: calculate total
FOR each subject column: calculate average
FIND maximum student total
FIND minimum subject average
DISPLAY totals, averages, top student, weakest subject
STOP
```

Python source code

```
students = ["Asha", "Bharat", "Charan"]
subjects = ["Python", "Maths", "Physics"]
marks = [
    [88, 76, 91],
    [72, 85, 70],
    [95, 89, 84]
]

totals = [sum(row) for row in marks]
averages = []
for col in range(len(subjects)):
    subject_total = 0
    for row in marks:
        subject_total += row[col]
    averages.append(subject_total / len(marks))

top_index = totals.index(max(totals))
weakest_index = averages.index(min(averages))

print("Student totals:")
for i, name in enumerate(students):
    print(f"{name}: {totals[i]}/300")
print("\nSubject averages:")
for i, subject in enumerate(subjects):
    print(f"{subject}: {averages[i]:.2f}")
print(f"Top performer: {students[top_index]}")
print(f"Weakest-average subject: {subjects[weakest_index]}")
```

Test case

Asha [88,76,91]; Bharat [72,85,70]; Charan [95,89,84]

Sample output

```
Student totals:
Asha: 255/300
Bharat: 227/300
Charan: 268/300

Subject averages:
Python: 85.00
Maths: 83.33
Physics: 81.67
Top performer: Charan
```

```
Weakest-average subject: Physics
```

Conclusion

The program uses nested lists and loops to summarize student performance by student and by subject.

Food Waste Monitoring System

Problem statement

Compare meals prepared, meals served, and leftovers across several days; calculate waste percentages and flag days that exceed an acceptable limit.

Objectives

• Calculate leftovers.
• Calculate waste percentage for each day.
• Identify days exceeding the waste threshold.

Assumptions

• Waste percentage = leftovers / prepared meals × 100.
• Acceptable waste limit is 10%.
• Prepared meals must be greater than zero and served meals cannot exceed prepared meals.

Algorithm

1. Store daily prepared and served counts.
2. Validate each record.
3. Calculate leftovers and waste percentage.
4. Flag records over 10%.
5. Calculate overall waste percentage.

Pseudocode

```
START
STORE daily prepared and served values
FOR each day
  IF prepared <= 0 OR served < 0 OR served > prepared: report invalid record
  ELSE leftovers = prepared - served
       waste_pct = leftovers / prepared * 100
       IF waste_pct > 10: flag day
CALCULATE overall waste percentage
DISPLAY report
STOP
```

Python source code

```
days = [
    ("Monday", 200, 185),
    ("Tuesday", 220, 190),
    ("Wednesday", 180, 175),
    ("Thursday", 210, 180)
]
limit = 10
total_prepared = 0
total_leftovers = 0

print("Food Waste Report (limit: 10%)")
for day, prepared, served in days:
    if prepared <= 0 or served < 0 or served > prepared:
        print(f"{day}: Invalid data")
        continue
    leftovers = prepared - served
    percentage = leftovers / prepared * 100
    total_prepared += prepared
    total_leftovers += leftovers
    status = "EXCEEDS LIMIT" if percentage > limit else "Within limit"
    print(f"{day}: prepared={prepared}, served={served}, leftovers={leftovers}, waste={percentage:.2f}% - {status}")

overall = total_leftovers / total_prepared * 100
print(f"Overall leftovers: {total_leftovers}")
print(f"Overall waste percentage: {overall:.2f}%")
```

Test case

Monday 200/185; Tuesday 220/190; Wednesday 180/175; Thursday 210/180

Sample output

```
Food Waste Report (limit: 10%)
Monday: prepared=200, served=185, leftovers=15, waste=7.50% - Within limit
Tuesday: prepared=220, served=190, leftovers=30, waste=13.64% - EXCEEDS LIMIT
Wednesday: prepared=180, served=175, leftovers=5, waste=2.78% - Within limit
Thursday: prepared=210, served=180, leftovers=30, waste=14.29% - EXCEEDS LIMIT
Overall leftovers: 80
Overall waste percentage: 9.88%
```

Conclusion

The program calculates daily and overall food waste and highlights days that exceed the allowed threshold.